DECORATORS

1. What Is a Decorator?

A decorator is a function that takes another function and returns a function with added or changed behaviour. It lets us add a reusable feature without editing the original function's code.

The situation: add messages around a function

```
def calculate():
    print("Performing calculation...")
    print(10 + 20)

calculate()
```

Imagine several functions need a message before they run and another message after they finish. Copying those messages into every function creates repetition. A decorator lets us place that extra behaviour in one reusable place.

Build a simple decorator

```
def my_decorator(func):
    def wrapper():
        print("Function execution started")

        func()  # Run the original function

        print("Function execution completed")

    return wrapper


def calculate():
    print("Performing calculation...")
    print(10 + 20)


calculate = my_decorator(calculate)
calculate()
```

Read the steps:

calculate is passed to my_decorator as the argument func.
my_decorator defines a new function called wrapper.
wrapper prints a message, calls the original function through func(), and prints another message.
my_decorator returns wrapper. The name calculate is then assigned to that returned function.
Calling calculate() now runs wrapper, which also runs the original calculation.

2. The @ decorator syntax

```
def my_decorator(func):
    def wrapper():
        print("Function execution started")
```

```
        func()
        print("Function execution completed")
    return wrapper


@my_decorator
def calculate():
    print("Performing calculation...")
    print(10 + 20)


calculate()
```

Remember @my_decorator above a function is shorthand for calculate = my_decorator(calculate). The decorator is applied when the function is defined.

A practical example: measure execution time

```
import time

def measure_time(func):
    def wrapper():
        start = time.time()
        func()
        end = time.time()
        print("Execution time:", end - start, "seconds")
    return wrapper


@measure_time
def do_work():
    total = 0
    for number in range(100000):
        total += number
    print("Work completed")


do_work()
```

The decorator measures how long do_work takes, without placing timing statements inside do_work itself. The measured time varies by computer and execution conditions.

3. Decorator with multiple arguments

The simple wrapper above works for functions that take no arguments. In practical code, decorators are often written with *args and **kwargs so the wrapper can pass along different positional and keyword arguments.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before")
        result = func(*args, **kwargs)
        print("After")
        return result
    return wrapper
```

```
@my_decorator
def greet(name):
    print("Hello", name)


greet("Asha")
```

Here, *args collects positional arguments and **kwargs collects keyword arguments. The wrapper forwards them to the original function. Returning result also preserves the original function's return value.

4. Decorators applications

Logging: record when a function is called.
Timing: measure how long a function takes.
Authentication and permissions: check whether a user is allowed to proceed.
Caching: reuse a previously calculated result.
Framework features: Python frameworks use decorators to register routes or mark special functions.

5. Quick Recap

A decorator is a function that accepts another function as an argument.
A decorator generally defines an inner function called a wrapper.
The wrapper executes additional functionality before and/or after calling the original function.
The decorator returns the wrapper function.
The @decorator_name syntax is a convenient way of applying a decorator to a function.

One-line definition for students A decorator is a function that takes another function as input, adds additional functionality to it, and returns the modified function without changing its original code.

6. Practice Exercises – Python Decorators

Exercise 1 – Before and After Execution: Create a decorator that displays "Starting function" before executing the original function and "Function completed" after its execution. Apply it to a function that displays a welcome message.

Exercise 2 – Function with One Argument: Create a decorator that displays a message before and after execution. Apply it to a function that accepts a student's name as an argument and displays a welcome message.

Exercise 3 – Function with Two Arguments: Create a decorator that displays "Calculating..." before execution. Apply it to a function that accepts two numbers and displays their product.

Exercise 4 – Using *args and **kwargs: Create a decorator using *args and **kwargs. Apply it to functions that accept different numbers of positional and keyword arguments. Test both types of function calls.